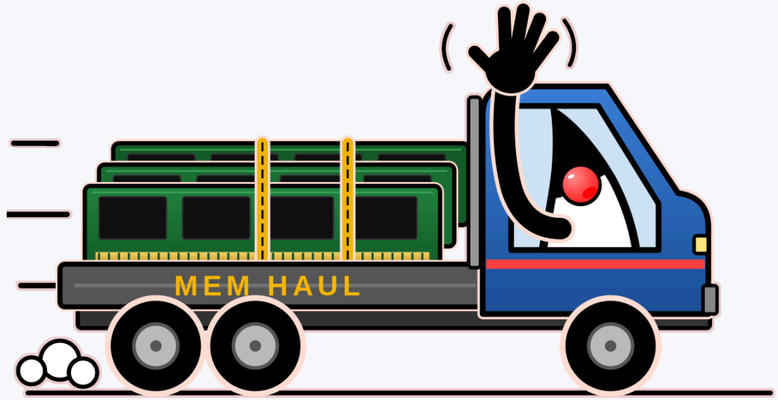
Who Moved My Heap?

Explore the Java heap for unpleasant surprises, and fix them with the right patterns, tools and runtime options.

Content and code

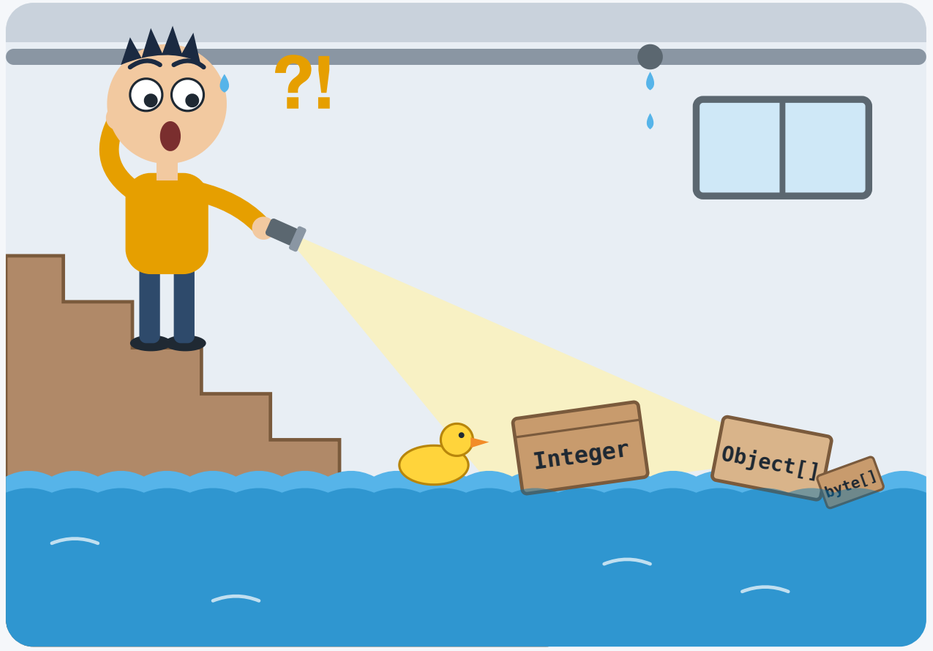
github.com/c-guntur/who-moved-my-heap



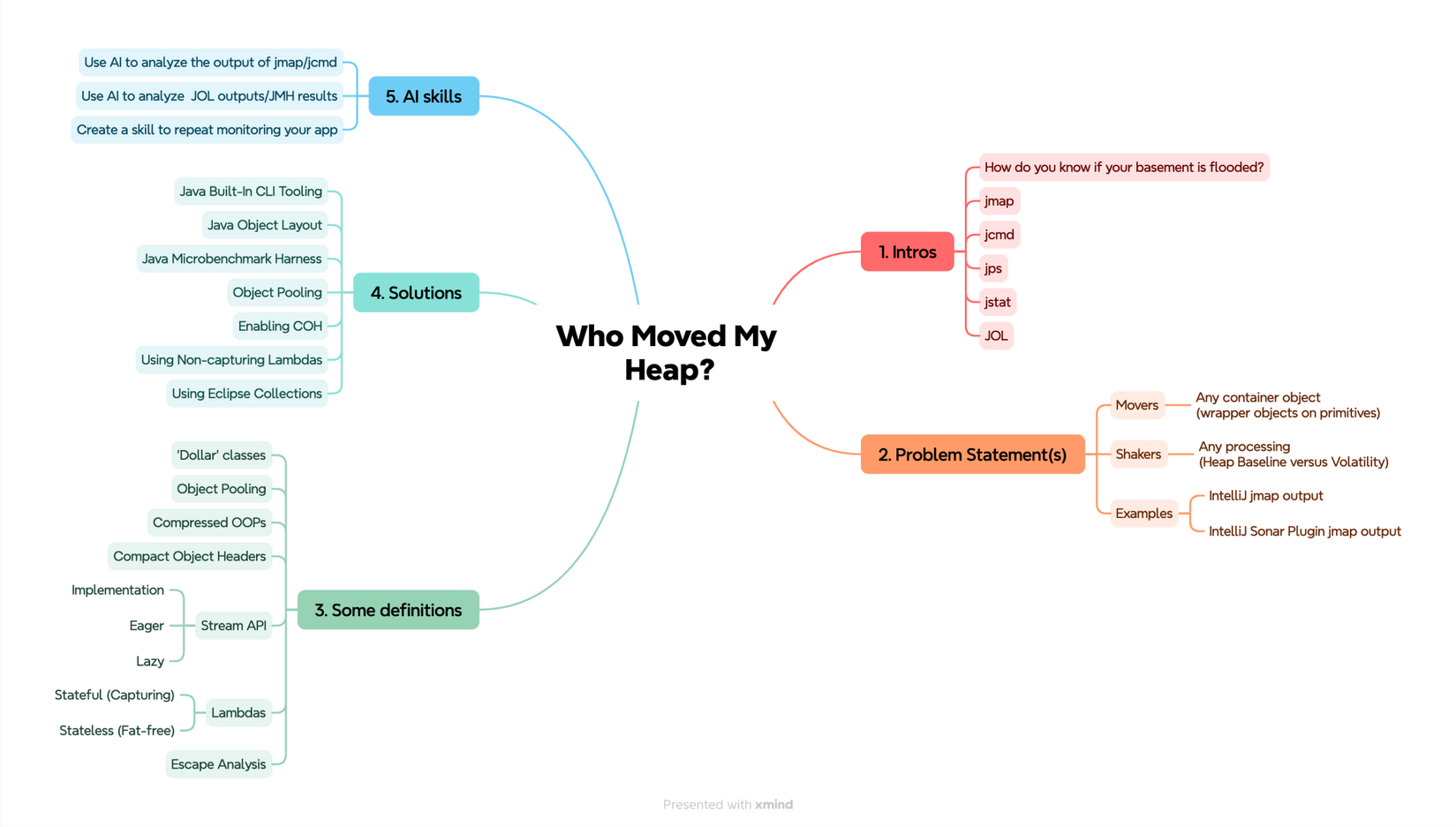
Opinions are personal and do not represent the views or opinions of the speakers' employer.

A question for you

How do you find out if your basement is flooded?



The map of this talk



Index

	Section	What it covers	Min
01	Intro	A few JDK tools and a library for looking at the heap	5
02	Problem statement	What JDK collections and streams leave on the heap	8
03	Definitions	The vocabulary behind who moved my heap	9
04	Solutions	Tools, runtime options and code that move less heap	10
05	AI skills	Let an assistant read the heap with you	4
06	Wrap-up	Is your basement flooded?	2
07	Appendix	Everything that did not fit in 40 minutes	

WHO MOVED MY HEAP · 01 INTRO

Intro

A few JDK tools and a library for looking at the heap

Intro | Problem | Definitions | Solutions | AI skills | Wrap-up

01

Why look at the heap?

A few JDK tools and one library show what is on the heap of a running JVM.

- Garbage is invisible in the code, but obvious in a class histogram.
- Fewer and smaller objects mean fewer GCs and more headroom for a spike.
- `jps` finds the JVM; `jmap` and `jcmd` look inside it; `jstat` watches its GCs.
- JOL, a library, measures single objects; garbage per call (JMH) comes with the solutions.

`jps`: which JVM is that?

`jps` lists the running Java processes of the current user, so you can find the pid the other tools need.

```
$ jps -l
674  Sleep.java
2310 org.apache.maven.wrapper.MavenWrapperMain
2488 jdk.jcmd/sun.tools.jps.Jps
```

- Each line is a pid and a main class or JAR; `-l` prints the full name.
- `-v` adds the JVM flags, which shows whether compact object headers are on.

`jmap`: what is on the heap?

`jmap` attaches to a running JVM and prints its objects, class by class.

```
jmap -histo:live <pid>                          # reachable objects only (runs a full GC first)
jmap -histo <pid>                               # every object, garbage included
jmap -dump:live,format=b,file=heap.hprof <pid>   # full heap dump for an analyzer
```

- A histogram is cheap and small; a dump holds every object and field value.
- `:live` forces a Full GC, so insights into existing garbage are lost.

Reading a class histogram


```
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:         15430         698576  [B (java.base@25.0.4.1)
   2:          3302         399272  java.lang.Class (java.base@25.0.4.1)
   3:         15273         366552  java.lang.String (java.base@25.0.4.1)
```

- `[B` on the first line is a `byte[]` - a String-backed array.
- `#bytes` divided by `#instances` is the average object size.
- Two histograms around a workload, subtracted, give the garbage it created.

`jcmd`: one tool, many commands

`jcmd` sends a diagnostic command to a running JVM; it covers most of what `jmap` and `jinfo` did.

```
jcmd                                # list the running JVMs and their pids
jcmd <pid> help                     # list the commands that JVM understands
jcmd <pid> GC.class_histogram       # the same histogram as jmap -histo:live
jcmd <pid> GC.class_histogram -all    # includes unreachable objects, no full GC
jcmd <pid> VM.flags                 # flags in effect, e.g. +UseCompactObjectHeaders
```

- `-all` counts garbage as well, so one call before and one after shows the allocation.
- `VM.flags` proves whether compact object headers were really on.

`jstat`: is the JVM busy collecting?

`jstat` samples the GC counters of a running JVM at an interval, without stopping it.

```
$ jstat -gcutil <pid> 1s 5                # five samples, one per second
     E       O    YGC     YGCT    FGC      GCT      (columns trimmed)
 32.79    2.42      7    0.078      0    0.078
  0.00    2.42      9    0.090      0    0.090
 65.57    2.42     10    0.095      0    0.095
 28.69    2.42     12    0.107      0    0.107
 95.08    2.42     13    0.113      0    0.113
```

- `E` and `O`: percent used of eden and old generation; eden fills up and empties again.
- `YGC` counts young GCs: here about 1.5 per second, all of them for garbage.

JOL: how big is one object?

Java Object Layout (JOL) is a small OpenJDK library that shows the header, fields, padding and size of real objects.

```
System.out.println(ClassLayout.parseInstance(Integer.valueOf(500)).toPrintable());

OFF  SZ   TYPE DESCRIPTION               VALUE
  0   8        (object header: mark)     0x0000000000000001 (non-biasable; age: 0)
  8   4        (object header: class)    0x0019e5d0
 12   4    int Integer.value             500
Instance size: 16 bytes
```

- A library, not a JDK tool: add `org.openjdk.jol:jol-core` to the build.

DEMO Demo: down to the basement

Find a running JVM and look at what it holds.

```
jps -l                                          # which JVM is that?
jcmd <pid> GC.class_histogram | head -12        # what is on its heap
jcmd <pid> VM.flags -all | grep UseCompactObjectHeaders
jstat -gcutil <pid> 1s 3                        # is it collecting right now?
```

- Look for `Integer`, `HashMap$Node`, `Object[]` and `$$Lambda` near the top.

Problem statement

What JDK collections and streams leave on the heap

Intro | **Problem** | Definitions | Solutions | AI skills | Wrap-up

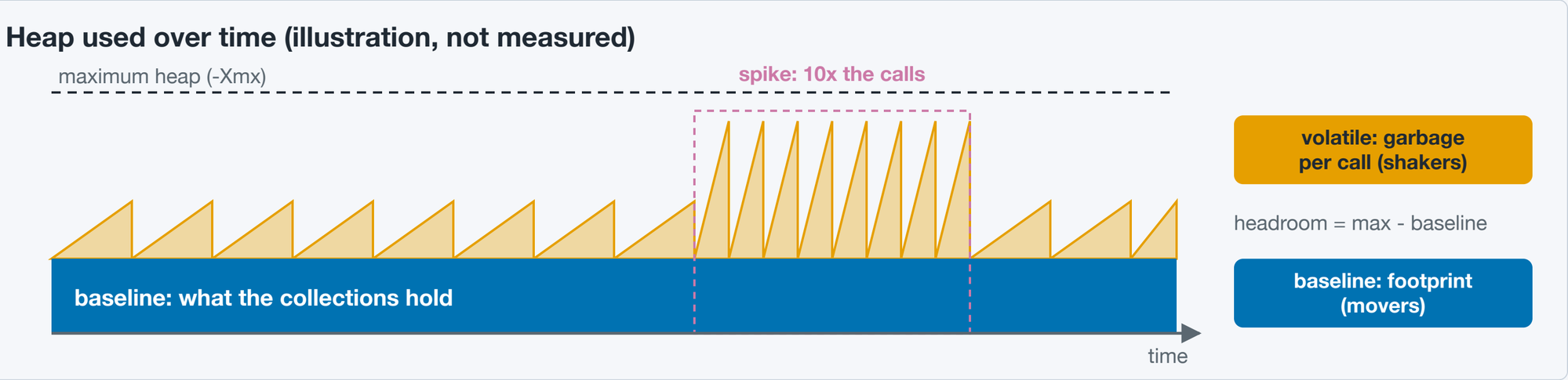
02

The problem in one slide

JDK Collections and Stream create objects on the heap that a developer may not know about.

- Boxed primitives and per-entry nodes inflate what a collection holds.
- Every stream call builds a pipeline of objects and drops it after.
- Amount of garbage produced depends on the effectiveness of the JVM’s **Escape Analysis**.
- **Capturing lambdas** have potential to leave garbage on the heap.

Movers and Shakers



- Headroom is what is left above the baseline for garbage.
- A spike multiplies the garbage rate; the baseline stays where it was.
- **Movers** focus on baseline memory.
- **Shakers** focus on volatile memory.

Why the volatile part is the risk

Stream garbage grows with calls, not with the number of elements.

- One JDK `anyMatch` call leaves 176 B, wherever the match is.
- At 10 M calls that is 1.76 GB of young-generation garbage.
- The same operation using Eclipse Collections allocates 0 B.

IntelliJ IDEA: what is on its heap

Top of `jmap -histo` against a running IntelliJ IDEA.

Screenshot to add

`jmap -histo <pid of IntelliJ IDEA>`

`assets/images/screenshot-problem-02-jmap-intellij.png`

SonarQube for IDE: what is on its heap

Top of `jmap -histo` against the SonarQube for IDE plugin process.

Screenshot to add

`jmap -histo <pid of the SonarQube for IDE process>`

`assets/images/screenshot-problem-02-jmap-sonarqube.png`

Boxing is a big cost

An `Integer` takes 16 B to hold a 4 B `int`, plus a 4 B reference to it.

1,000,000 elements, compact headers on	Retained	Per element
JDK `ArrayList<Integer>`, presized	20.0 MB	20 B
EC `IntArrayList` (primitive)	4.0 MB	4 B
JDK `HashSet<Integer>`	48.4 MB	48 B
EC `IntHashSet` (primitive)	8.4 MB	8 B
JDK `HashMap<Integer, Integer>`	48.4 MB	48 B
EC `IntIntHashMap` (primitive)	16.8 MB	17 B

Every `HashMap` entry is one more object

`HashMap` wraps each entry in a `HashMap$Node`.

- Each node instance is 24 B with **Compact Object Headers (COH)** on, 32 B with COH off.
- A `HashSet` contains a `HashMap`, thus also inherits the problem.
- Thus, `Node` plus `Integer` (16 B) ≈ 40 (or 48 B COH off) of heap per element to hold one `int`.
- EC `UnifiedSet` keeps the elements in its table: 8.4 MB, 4x less.

A stream is an object graph before it runs

An unevaluated pipeline already holds objects, and every operation adds a stage.

Pipeline on an empty list, compact headers on	JDK `Stream`	EC `LazyIterable`
start: `stream()` / `asLazy()`	80 B	16 B
+ `filter` / `select`	144 B	24 B
+ `map` / `collect`	144 B	24 B
+ `distinct`	128 B	16 B
3 steps: filter, map, mapToLong	272 B	96 B

Every stream call builds a pipeline and drops it

Bytes and objects per call, match at element 500, compact headers on.

Operation	JDK Object Count	JDK Stream Size	Equivalent EC Size
`anyMatch` / `anySatisfy`	7	176 B	0 B
`filter().findFirst()` / `detect`	**8**	**216 B**	**0 B**
`filter().count()` / `count`	8	224 B	0 B

- The stream's own cost is per call: the same at element 1 as at 1000.
- EC versions: capturing or fat-free, eager or lazy, all 0 B.
- ⇒ **Let's dig deeper into the JDK `filter().findFirst()` to look at the 216 B allocation**.

Where the bytes come from

`list.stream().filter(p).findFirst()`, one call, match at element 500.

Origin	Objects	Bytes
Stream source: `ReferencePipeline$Head`, spliterator	2	80 B
`filter` stage and its sink	2	72 B
terminal op: `FindSink`, `Optional`	2	32 B
caller lambda `Integer.valueOf(test)::equals`	1	16 B
`Integer` box, outside the cache	1	16 B
Total per call	**8**	**216 B**

From bytes per call to spikes

Bytes per call are measured; rates and spikes are derived.

Element 500	B per call	MB/s at 10 M calls/s	10 s spike
JDK `anyMatch`	176	1,760	17.6 GB
JDK `findFirst`	216	2,160	21.6 GB
JDK `count`	224	2,240	22.4 GB
EC, every version	0	0	0

Problem statement

JDK Collections and Stream impact the heap.

- **Baseline**: Collections have Boxed primitives / extra classes like `HashMap$Node`.
- **Volatile**: Streams produce additional 160 B - 224 B of pipeline garbage on every stream call.
- **Volatile**: Streams do not benefit from **Escape Analysis**.

- **Statement**: Larger baselines, increased volatility risk instability if there are spikes.

Definitions

The vocabulary behind who moved my heap

03

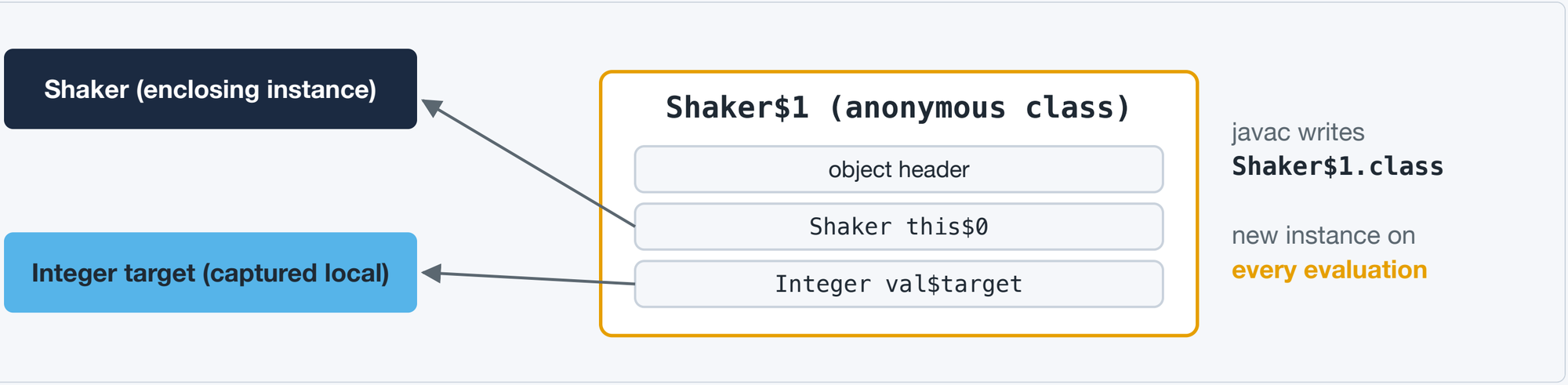
Seven terms

The words the solutions need, one slide each.

#	Term	In one line
01	Compressed OOPs	4-byte references on a 64-bit JVM
02	Compact Object Headers	One 8-byte header word (JDK 25+)
03	"Dollar" classes	Anonymous classes and their `Outer$1` names
04	Escape analysis	When the JIT removes *unescaped* allocations
05	Stream API	Lazy intermediate, eager terminal operations
06	Lambdas	Capturing vs fat-free
07	Immutable Data Object pooling	De-duplicating Objects

"Dollar" classes

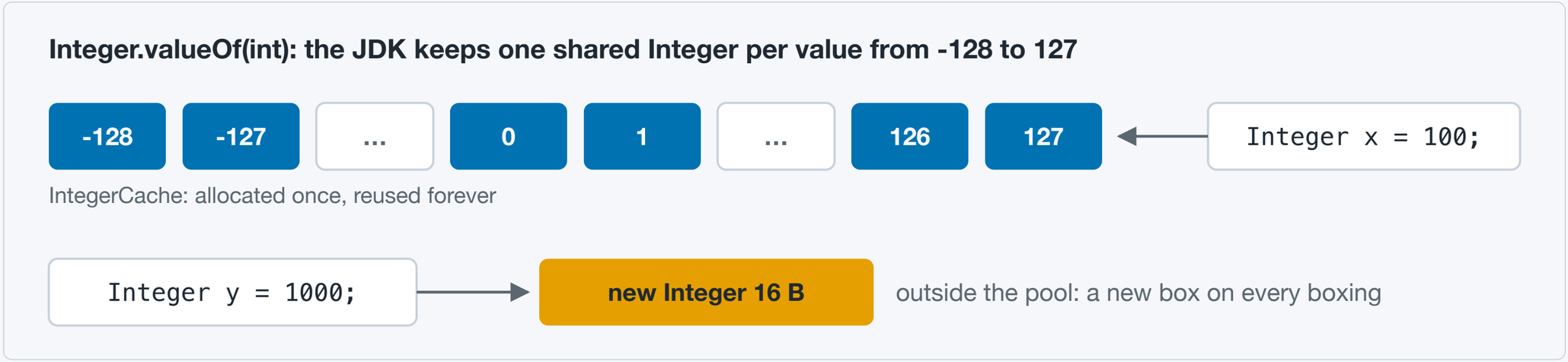
An anonymous class has no name in the code; `javac` names it `Outer$1`.



- A new instance on every evaluation, even when nothing is captured.
- In an instance method it always holds the enclosing `this` (`this$0`).

Object pooling

A pool hands out the same objects again instead of allocating new ones.



- The JDK pools small `Integers`: autoboxing calls `Integer.valueOf(int)`.
- -128 to 127 come from the cache; any other value is a new 16 B object.

Compressed OOPs

A 4-byte offset that HotSpot stores instead of an 8-byte reference, for heaps under 32 GB.

A reference field: uncompressed vs compressed

64-bit address — **8 bytes**

compressed OOP — **4 bytes** — half the size in every field and Object[] slot

```
address = heap base + (offset << 3)
```

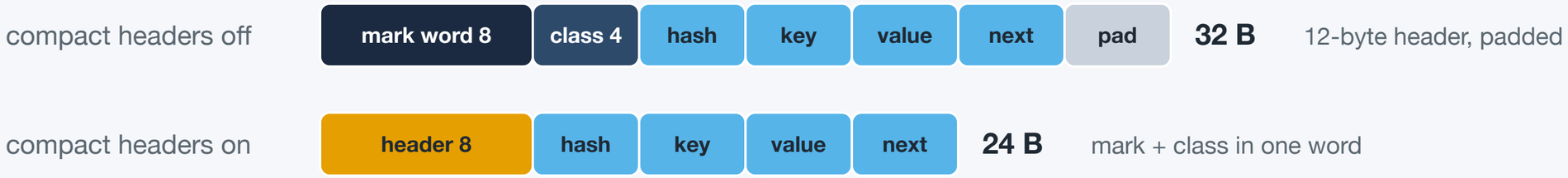
2^32 × 8-byte alignment
= heaps up to 32 GB

- Every reference field and `Object[]` slot shrinks from 8 to 4 bytes.
- On by default (`-XX:+UseCompressedOops`).

Compact object headers

The object header shrinks from 12 bytes to one 8-byte word (JEP 519, JDK 25).

HashMap$Node layout (box width = bytes)

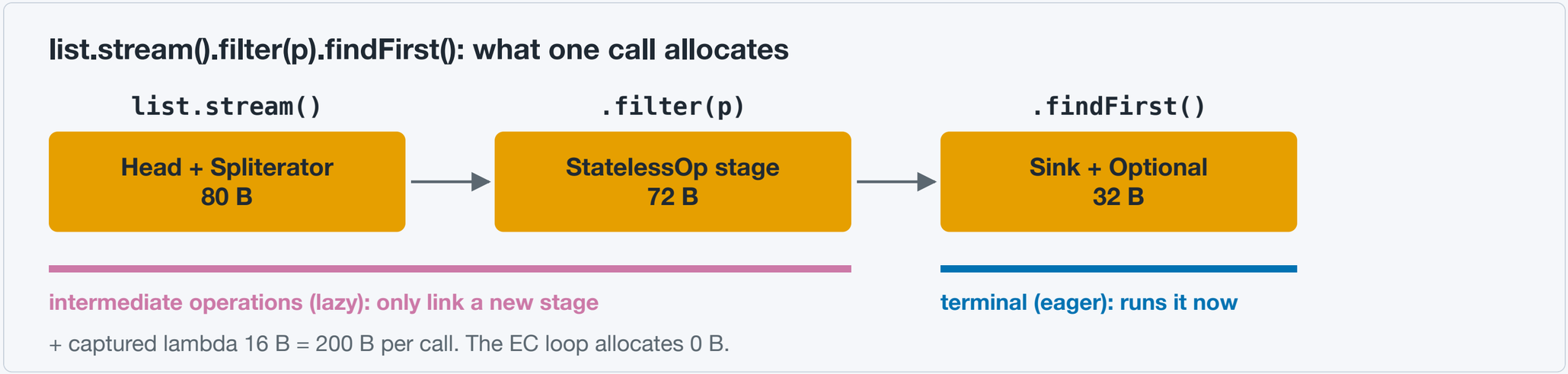


1M HashMap entries: 56.4 MB → 48.4 MB. An Integer stays 16 B either way (12 B rounds up to 16).

- Off by default: `-XX:+UseCompactObjectHeaders`.
- An object only shrinks if it then fits a smaller 8-byte boundary.

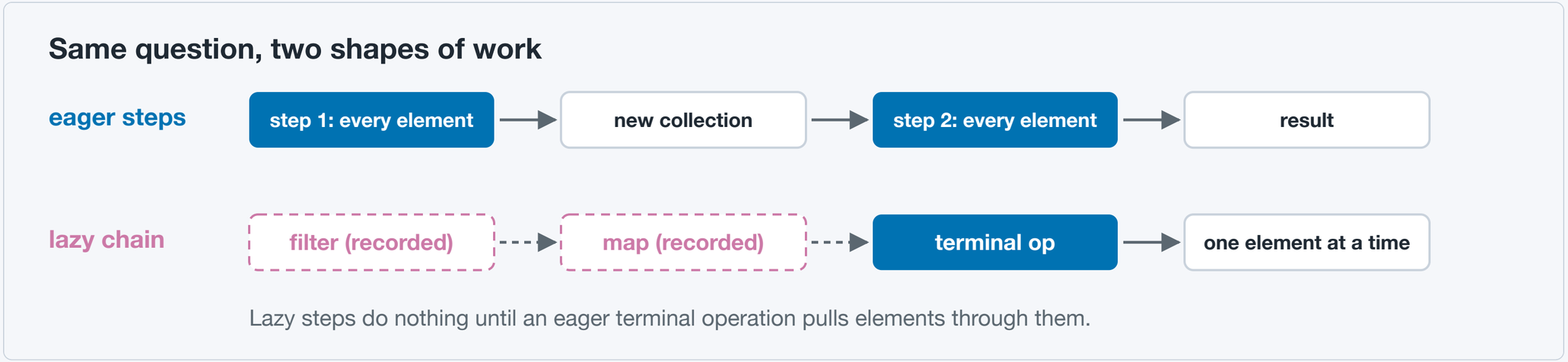
Stream API: what one call allocates

Intermediate operations (`filter`, `map`) are lazy; the terminal operation (`anyMatch`, `count`) is eager and runs the pipeline.



- Every call builds this graph again; escape analysis removes none of it.

Stream API: eager vs lazy



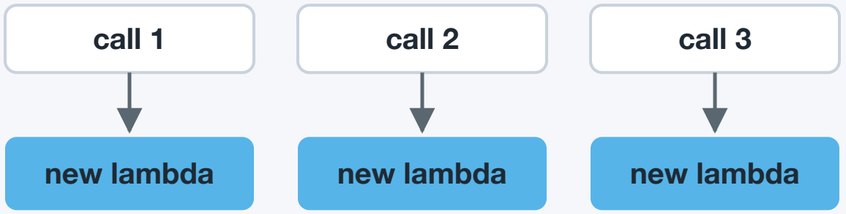
- **Eager**: runs when called and returns a result.
- **Lazy**: recorded; runs one element at a time when a result is needed.

Lambdas: capturing vs fat-free

A capturing lambda holds values from its scope; a fat-free one uses only its parameters.

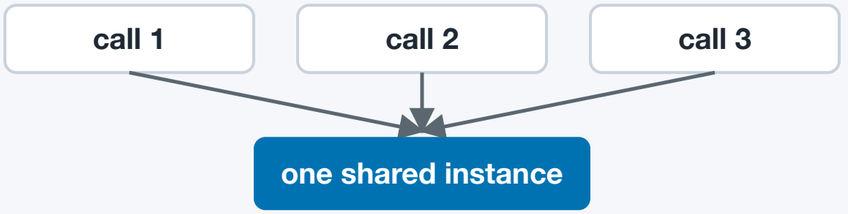
Three calls to the same method

`capturing: each -> each.equals(target)`



3 objects, each holding target

`fat-free: Object::equals, target`



target travels as an argument

- Fat-free: one instance, created once and reused on every call.
- Capturing: a new object that holds the captured values, per evaluation.

Escape analysis

A C2[1] JIT[2] check: when no one outside the method can see a new object, the allocation is removed.

After inlining, C2 asks: can anyone outside this method see the new object?



Escapes = stored in a field, returned, or passed to code that was not inlined. Turn it off: -XX:-DoEscapeAnalysis

- Works only on compiled, inlined code, after warm-up.

---

[1] C2: the HotSpot server compiler, the optimizing compiler for hot methods (C1 is the fast client compiler).

[2] JIT: just-in-time compiler, which turns frequently run bytecode into machine code while the program runs.

Solutions

Tools, runtime options and code that move less heap

04

Solutions at a glance

Solution	Use it to	Measured here
JDK CLI tooling	See what is on the heap, and what a workload drops	Measure first
Java Object Layout	Size one object or a whole collection	`HashMap$Node` 32 B to 24 B
Java Microbenchmark Harness	Count bytes and objects per call	176 B per `anyMatch`
Object pooling	Reuse costly objects, not small ones	`Integer` -128 to 127
Compact object headers	Shrink every header by 4 B	14% off a `HashMap`
Non-capturing lambdas	Share one lambda instance	32 B to 16 B per call (EA off)
Eclipse Collections	Primitives, and loops without a pipeline	48 B to 8 B per element; 0 B per call

Measure first: before and after

Two class histograms around a workload show what it allocated.

```
jps -l                                                    # find <pid>
jmap -histo <pid> | grep "Lambda\|ReferencePipeline\|CollectorImpl\|HashMap\|HashSet\|ArrayList\|Integer\|Long\|Double\|Float\|Pair\|Strir
# ... run the workload ...
jmap -histo <pid> | grep "Lambda\|ReferencePipeline\|CollectorImpl\|HashMap\|HashSet\|ArrayList\|Integer\|Long\|Double\|Float\|Pair\|Strir
```

- Focus on the big ticket memory occupiers.
- Subtract per class: that is how `prof.HistoProfiler` counts objects per call.

JOL: the layout of one object

Java Object Layout (JOL) reports how the JVM lays an object out: header, fields, padding and size.

```
import org.openjdk.jol.info.ClassLayout;
import org.openjdk.jol.info.GraphLayout;

System.out.println(ClassLayout.parseClass(String.class).toPrintable());

long bytes = GraphLayout.parseInstance(array).toFootprint();
```

- `ClassLayout` shows one object; `GraphLayout` adds up everything reachable from it.
- JOL is an OpenJDK project, added as a library; it is not part of the JDK.

JOL with compact object headers

`HashMap$Node`	Header	Fields	Size
Without CompactObjectHeaders	12 B	hash, key, value, next: 16 B	32 B (4 B padding)
With CompactObjectHeaders	8 B	the same 16 B	24 B

- Run the same JOL code with the flag off and on, and compare the sizes.
- The 4 B saved, only counts when the total drops below the next 8 B boundary.
- Note that Compact Object Headers are available from Java 25, but are not on by default.
- You can turn on Compact Object Headers with a CLI flag `-XX:+UseCompactObjectHeaders`.
- Compact Object Headers will be on by default with Java 27 and beyond.

DEMO Demo: one map entry, headers off and on


```
Class<?> node = Class.forName("java.util.HashMap$Node");
System.out.println(ClassLayout.parseClass(node).toPrintable());
```

```
java -cp jol-core-0.17.jar -XX:-UseCompactObjectHeaders NodeLayout.java   # 32 bytes
java -cp jol-core-0.17.jar -XX:+UseCompactObjectHeaders NodeLayout.java   # 24 bytes
```

- Same fields, 4 B less header, and the 4 B alignment gap is gone.

JMH: bytes per call, not only speed

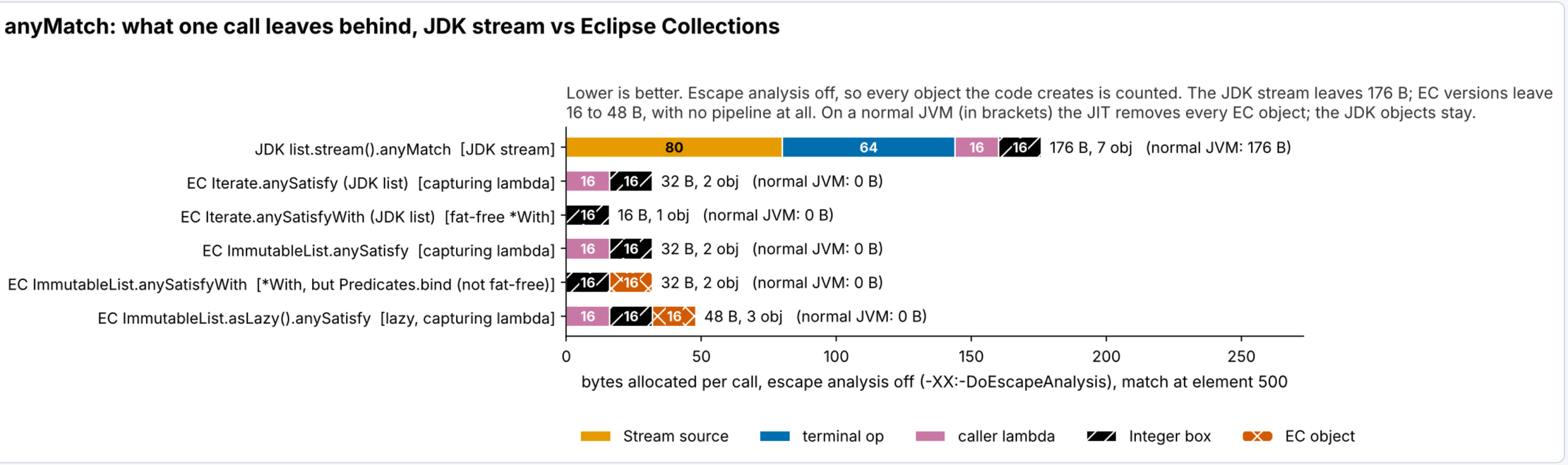

```java
@Benchmark
public boolean jdkAnyMatch()
{
    return jdkList.stream().anyMatch(Integer.valueOf(this.test)::equals);
}

@Benchmark
public boolean ecAnySatisfyWith()
{
    return Iterate.anySatisfyWith(jdkList, Integer::equals, this.test);
}
```

- `-prof gc` reports `gc.alloc.rate.norm`: bytes allocated per call.
- `-prof prof.HistoProfiler` counts the objects per call.

What one `anyMatch` call leaves behind

Escape analysis off: every object each version creates, counted.



Object pooling: pool resources, not small objects

A pool pays off for costly objects; for small ones, allocate fewer instead.

- Pool threads, connections and direct buffers.
- The JDK already pools `Integer` -128 to 127 through `Integer.valueOf`.
- For small objects: presize collections, use primitives, avoid boxing.
- A pooled object lives long, gets promoted, and needs locking.

Enabling compact object headers


```
java -XX:+UseCompactObjectHeaders -jar app.jar
jcmd <pid> VM.flags -all | grep UseCompactObjectHeaders     # check that it is on
```

Retained size	Off	On	Saved
`HashMap<Integer, Integer>`, 1 M entries	56.4 MB	48.4 MB	14%
`ArrayList<Integer>`, 1 M, presized	20.0 MB	20.0 MB	0%
`Stream.filter().map().mapToLong()`	304 B	272 B	11%

- One flag, no code change: it shrinks nodes, not boxes.

Non-capturing lambdas


```
// capturing: a new lambda on every call, holding the value
Iterate.anySatisfy(list, Integer.valueOf(test)::equals);

// fat-free: one constant lambda, the value is a parameter
Iterate.anySatisfyWith(list, Integer::equals, test);
```

- Escape analysis off: 32 B capturing, 16 B fat-free (only the `Integer` box).
- Move captured values into parameters: the `*With` methods.

Not all `anySatisfyWith()` methods in EC are non-capturing (fat-free).

Eclipse Collections: primitives and no pipeline

	JDK	Eclipse Collections
1 M `int`s in a list	20.0 MB `ArrayList<Integer>`	4.0 MB `IntArrayList`
1 M `int`s in a set	48.4 MB `HashSet<Integer>`	8.4 MB `IntHashSet`
`anyMatch`, match at 500	176 B per call	0 B `anySatisfy`
`filter().count()`	224 B per call	0 B `count`

- Primitive collections drop the boxes; eager methods loop without a pipeline.
- `*With` methods keep the lambda fat-free.

AI skills

Let an assistant read the heap with you

05

AI reads a class histogram

Paste two histograms and ask what moved, not what a histogram is.

```
Here are two `jcmd GC.class_histogram -all` outputs, before and after one request.
Which classes grew most, in instances and in bytes?
Which are boxes, map nodes, arrays or lambdas, and what code likely made them?
```

- A histogram holds class names and counts; a heap dump holds your data, keep it local.
- Ask for the arithmetic, then check two rows yourself.

AI reads JOL and JMH results

Raw JMH and JOL output is long; an assistant turns it into a comparison.

- Give it the raw files (`-rf text`, JOL printouts), not a summary.
- Ask for bytes per call by variant: headers off and on, escape analysis off.
- Ask which numbers are measured and which are derived.
- Every number on these slides traces back to a raw file under `generated/runs`.

A skill to repeat the check


```
---
name: heap-check
description: Compare class histograms of a running JVM before and after a workload
---
1. Run `jps -l` and pick the pid of the app.
2. Run `jcmd <pid> GC.class_histogram -all` before and after the workload.
3. Subtract per class; list the top 10 by bytes.
4. Flag boxes, `HashMap$Node`, `Object[]` and `$$Lambda` classes.
5. Compare with the last baseline in `heap-baseline.txt`.
```

- Run it every release, not only when the basement floods.

DEMO Demo: who moved my heap?

The histogram from the first demo, pasted into an AI assistant.

```
This is `jcmd <pid> GC.class_histogram` of a running JVM.
Who moved my heap? Name the top 5 classes by bytes, say which are
boxes, nodes, arrays or lambdas, and what I could change.
```

Is your basement flooded?

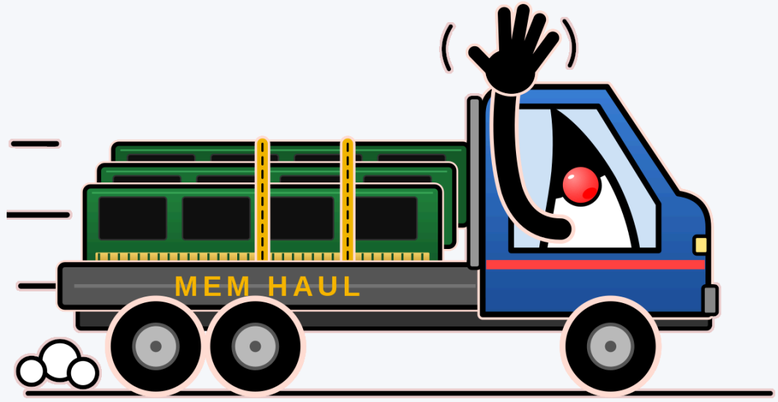
You only know if you go down and look.

- **Look**: `jps`, `jmap` and `jcmd`.
- **Investigate**: use JOL and JMH
- **Baseline**: primitives, pre-sized collections, compact object headers.
- **Volatile**: fat-free lambdas and loops instead of a stream per call.
- **Repeat**: a skill or a script that checks every release.

Thank you. Questions?

Content and code

github.com/c-guntur/who-moved-my-heap



Appendix

Everything that did not fit in 40 minutes

07

What is in the appendix

Appendix

#	Part	What it holds
00	Timing plan	The 40 minutes, section by section
01	Intro	`jcmd` command table, `jstat -gc` and options, more JOL output, adding JOL
02	Problem statement	Two heaps, histograms, slack, compact headers, escape analysis
03	Definitions	The full definitions: what, in code, inside the JDK
09	References	Every reference slide, by section, JOL included

Timing plan

Appendix

Start	Min	Section	Demo or audience
00:00	2	Opening	Question: is your basement flooded?
02:00	5	Intro: `jps`, `jmap`, `jcmd`, `jstat`, JOL	Demo 1: class histogram
07:00	8	Problem statement	-
15:00	9	Definitions	Show of hands: compact headers
24:00	10	Solutions	Demo 2: JOL, headers off and on
34:00	4	AI skills	Demo 3: ask who moved my heap
38:00	2	Wrap-up	Answer the opening question

`jcmd` commands for the heap

Appendix

Command	Shows
`GC.class_histogram`	Instances and bytes per class
`GC.heap_info`	Heap regions: total, committed and used
`GC.heap_dump <file>`	HPROF dump for a heap analyzer
`GC.run`	Requests a full GC
`VM.flags`	The JVM flags in effect
`VM.native_memory`	Native memory by area (needs `-XX:NativeMemoryTracking`)
`Thread.print`	Thread dump

Watching garbage with `jstat -gc`

Appendix

`-gc` shows the same counters in KB, so eden size and GC count give a rough allocation rate.

```
$ jstat -gc <pid> 1s 3                    (columns trimmed, sizes in KB)
       EC           EU           OU      YGC     YGCT
 124928.0      35840.0      2180.4       14    0.120
 125952.0     115712.0      2180.4       15    0.125
 130048.0      61440.0      7300.4       17    0.135
```

- `EU` climbs towards `EC`, a young GC runs, and eden starts again: a sawtooth.
- Eden capacity times young GCs per second is roughly the allocation rate: about 125 MB × 1.5/s ≈ 190 MB/s.
- `jstat -gccause` adds the cause of the last GC, here `G1 Evacuation Pause`.

Choosing a `jstat` option

Appendix

Option	Shows
`-gcutil`	Percent used per generation, GC counts and times
`-gc`	The same in KB: capacities and used sizes
`-gccause`	`-gcutil` plus the cause of the last and the current GC
`-gccapacity`	Minimum, maximum and current capacity per generation
`-gcnew`	Young generation: survivor spaces and tenuring
`-class`	Classes loaded and unloaded

JOL: one `Integer`, compact headers on

Appendix

With `-XX:+UseCompactObjectHeaders` the class pointer moves into the 8 B mark word.

```
java.lang.Integer object internals:
OFF  SZ   TYPE DESCRIPTION               VALUE
  0   8        (object header: mark)     0x001ac00000000001 (Lilliput)
  8   4    int Integer.value             500
 12   4        (object alignment gap)
Instance size: 16 bytes
Space losses: 0 bytes internal + 4 bytes external = 4 bytes total
```

- The header shrinks from 12 B to 8 B, but the object still rounds up to 16 B.
- The 4 B saved becomes an alignment gap: a box is a box.

JOL: the footprint of a small list

Appendix

```
List<Integer> list = new ArrayList<>(List.of(500, 501, 502));
System.out.println(GraphLayout.parseInstance(list).toFootprint());
```

Object	Count	Headers off	Headers on
`Object[]` (the backing array)	1	32 B	24 B
`Integer`	3	48 B	48 B
`ArrayList`	1	24 B	24 B
Total	5	104 B	96 B

- `GraphLayout` follows every reference, so the boxes count too: half the bytes are `Integer`.

Adding JOL to a project

Appendix

```
<dependency>
    <groupId>org.openjdk.jol</groupId>
    <artifactId>jol-core</artifactId>
    <version>0.17</version>
</dependency>
```

```
java -XX:+EnableDynamicAgentLoading -Djdk.attach.allowAttachSelf=true ...   # let JOL attach to its own JVM
java -jar jol-cli.jar internals java.util.HashMap                           # no code: the command line tool
```

- `jol-cli` is built from the openjdk/jol repository; it also has `footprint` and `heapdump-*` modes.
- jol-samples in the same repository walk through headers, alignment, arrays and more.

Two heaps in one

Appendix

A heap has a baseline that stays and a volatile part that churns.

- Baseline: the live objects your collections hold between calls.
- Volatile: short-lived garbage that each call allocates and drops.
- The baseline sets the floor; the volatile part sets how often GC runs.
- Both have to fit under `-Xmx` at the busiest moment.

Taking the histograms

Appendix

Two everyday JVMs on a developer laptop, and what they hold.

```
jps -l                            # find the pids
jmap -histo <pid> | head -15      # IntelliJ IDEA
jmap -histo <pid> | head -15      # SonarQube for IDE
```

- Without `:live` the histogram also counts garbage not yet collected.
- Look for `Integer`, `HashMap$Node`, `Object[]` and `$$Lambda` near the top.

Growing a collection leaves slack and garbage

Appendix

A list grown from empty is bigger than a presized one, and it dropped arrays on the way.

- `ArrayList` grows by 1.5x: up to 7% unused capacity in this run.
- Every array it outgrew became garbage; JOL only sees the final list.
- `HashMap` tables double: 46 to 50.5 B per entry, depending on n.
- Presize when the size is known: `new ArrayList<>(n)`, `HashMap.newHashMap(n)`.

Compact headers shrink nodes, not boxes

Appendix

`-XX:+UseCompactObjectHeaders` saves 4 B per header, but objects round up to 8 B.

Retained size	Headers off	Headers on	Saved
`HashMap<Integer, Integer>`, 1 M entries	56.4 MB	48.4 MB	14%
`HashSet<Integer>`, 1 M elements	56.4 MB	48.4 MB	14%
`ArrayList<Integer>`, 1 M, presized	20.0 MB	20.0 MB	0%
`Stream.filter().map().mapToLong()`	304 B	272 B	11%
one `Integer`	16 B	16 B	0%

Garbage you cannot read from the code

Appendix

The same line of code leaves different garbage under different conditions.

- `Integer.valueOf(1)` is cached; `Integer.valueOf(500)` costs 16 B per call.
- Escape analysis removes every EC object, and none of the stream's.
- Compact headers save 16 to 24 B per call; the objects stay.
- `anyMatch(p)` is 40 B cheaper than `filter(p).findFirst()`.

Escape analysis is not a promise

Appendix

With `-XX:-DoEscapeAnalysis` each lambda shape shows what it creates.

`anyMatch`, match at element 500	**EA on**	**EA off**
JDK `list.stream().anyMatch(…)`	176 B	176 B
EC lazy `asLazy().anySatisfy(…)`	0 B	48 B
EC capturing `Iterate.anySatisfy(…)`	0 B	32 B
EC fat-free `Iterate.anySatisfyWith(…)`	0 B	16 B

- The JIT removes allocations only in code it inlines.
- Turning it off is an upper bound, not a typical JVM.

Fat-free lambdas leave less behind

Appendix

```
// capturing: a new lambda on every call, holding the value
Iterate.anySatisfy(list, Integer.valueOf(test)::equals);

// fat-free: one constant lambda, the value is a parameter
Iterate.anySatisfyWith(list, Integer::equals, test);

// JDK stream: the pipeline is built whatever the lambda
list.stream().anyMatch(Integer.valueOf(test)::equals);
```

- Escape analysis off: 32 B capturing, 16 B fat-free, 176 B stream.
- Less garbage per call keeps the headroom free for the next spike.

What is a "dollar" class?

Appendix

An anonymous class is declared and created in one expression, without a name.

- `javac` still names it: `Outer$1`, `Outer$2`, …
- Each one gets its own `Outer$1.class` file.
- Named inner and local classes use `$` names too.

"Dollar" classes in code

Appendix

```
public class Shaker
{
    boolean hasMatch(List<Integer> list, Integer target)
    {
        return list.stream().anyMatch(new Predicate<Integer>()  // Shaker$1
        {
            @Override
            public boolean test(Integer each)
            {
                return each.equals(target);
            }
        });
    }
}
```

- Compare with lambdas (06): a non-capturing lambda is created only once.

What is object pooling?

Appendix

A pool keeps objects and hands them out again instead of allocating new ones.

- Callers return an object to the pool when they are done.
- **Worth it** for costly or limited resources: threads, connections, direct buffers.
- **Rarely worth it** for small, short-lived objects:
  - young-generation allocation is cheap;
  - pooled objects live long and need locking.

What is a compressed OOP?

Appendix

A 4-byte offset that HotSpot stores instead of an 8-byte reference.

- OOP = ordinary object pointer: a reference to a Java object.
- The offset is scaled by the 8-byte object alignment.
- On by default for heaps under 32 GB (`-XX:+UseCompressedOops`).

What is a compact object header?

Appendix

The object header shrinks from 12 bytes to one 8-byte word.

- A header holds the mark word (hash, lock, GC state) and a class pointer.
- Product feature in JDK 25 (JEP 519); experimental in JDK 24 (JEP 450).
- Off by default: `-XX:+UseCompactObjectHeaders`.

What is a stream?

Appendix

A sequence of elements processed by a pipeline: a source, intermediate operations, one terminal operation.

- Intermediate operations (`filter`, `map`) are **lazy**: they only record work.
- The terminal operation (`anyMatch`, `count`, `findFirst`) is **eager**: it runs the pipeline.
- Short-circuiting terminals stop at the first match.
- A stream can be used only once.

A pipeline in code

Appendix

```
boolean found = list.stream()                       // source
                    .filter(each -> each > 500)     // intermediate (lazy)
                    .findFirst()                    // terminal (eager)
                    .isPresent();
```

- Every call builds a new pipeline of objects.
- Nothing runs until `findFirst()`.

Inside: `ReferencePipeline`

Appendix

The internal class behind `Stream<T>`. Primitive versions: `IntPipeline`, `LongPipeline`, `DoublePipeline`.

- `stream()` creates a `Head` stage and a `Spliterator`.
- Each intermediate operation links one new stage (`StatelessOp` or `StatefulOp`).
- The terminal operation creates a `TerminalOp`, chains `Sink` objects and pushes elements through them.

Eager vs lazy in the JDK

Appendix

	Eager	Lazy
When work happens	At each call	At the terminal operation
In-between results	A new collection per step	None
Short-circuit	Per step	Across the whole chain
JDK examples	`anyMatch`, `count`, `findFirst`, `collect`, `removeIf`, `forEach`	`filter`, `map`, `sorted`

What is a lambda?

Appendix

An anonymous function that implements a functional interface such as `Predicate`.

- No `$` class file: `javac` emits an `invokedynamic` call.
- The runtime (`LambdaMetafactory`) builds the class on first use.
- Whether it allocates depends on what it captures.

Capturing (stateful) lambdas

Appendix

Uses a local, a parameter or `this` from the enclosing scope.

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return list.stream().anyMatch(each -> each.equals(target));  // new object per call
}
```

- Each evaluation creates a new object that holds the captured values.
- Captured locals must be effectively final.

Fat-free (non-capturing) lambdas

Appendix

Uses only its own parameters, constants and static members.

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return Iterate.anySatisfyWith(list, Object::equals, target);  // one shared instance
}
```

- One instance is created and then reused on every call.
- Extra values travel as arguments (the `*With` methods).

What is escape analysis?

Appendix

A C2 JIT check: after inlining, can anyone outside the method see the new object?

- **No**: the allocation is removed; its fields become locals (scalar replacement).
- **Yes**: it is allocated on the heap as usual.
- On by default; `-XX:-DoEscapeAnalysis` turns it off.

Escape analysis in code

Appendix

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return Iterate.anySatisfy(list, each -> each.equals(target));
}
```

- After inlining, the lambda and the loop stay local: 0 B allocated.
- The JDK stream escapes into its own stages: 160 to 224 B per call either way.

References: Intro

Appendix

Section	Reference
jmap	The `jmap` command (Java SE 25)
jcmd	The `jcmd` command (Java SE 25)
jps	The `jps` command (Java SE 25)
jstat	The `jstat` command (Java SE 25)
jmap, jcmd, jps, jstat	Troubleshooting Guide: Diagnostic Tools (Java SE 25)
JOL	See References: JOL, the next slide

References: JOL

Appendix

Topic	Reference
JOL	OpenJDK Code Tools: Java Object Layout (JOL)
Source, CLI	openjdk/jol: jol-core, jol-cli and jol-samples
Samples	jol-samples: headers, alignment, arrays and more
Library	Maven Central: `org.openjdk.jol:jol-core`
Headers	JEP 450: Compact Object Headers (Experimental), JDK 24
Headers	JEP 519: Compact Object Headers, JDK 25
Headers	OpenJDK Project Lilliput
Flag	The `java` command (Java SE 25): `-XX:+UseCompactObjectHeaders`

References: Baseline and volatile heap

Appendix

- HotSpot Virtual Machine Garbage Collection Tuning Guide (Java SE 25)
- Factors Affecting Garbage Collection Performance (Java SE 25)
- The `java` command: `-Xmx` and GC options (Java SE 25)

References: jmap in the wild

Appendix

- The `jmap` command (Java SE 25)
- The `jps` command (Java SE 25)
- Troubleshooting Guide: Diagnostic Tools (Java SE 25)

References: Movers

Appendix

- Java Object Layout (JOL), OpenJDK Code Tools
- `Integer.valueOf(int)` (Java SE 25 API)
- `HashMap` (Java SE 25 API)
- `ArrayList` (Java SE 25 API)
- The `java` command: `-XX:+UseCompactObjectHeaders` (Java SE 25)

References: Shakers

Appendix

- Package `java.util.stream` (Java SE 25 API)
- The Java Tutorials: Lambda Expressions
- Java HotSpot VM Performance Enhancements: Escape Analysis (Java SE 25)
- Java Magazine: Escape analysis in the HotSpot JIT compiler
- HotSpot Virtual Machine Garbage Collection Tuning Guide (Java SE 25)

References: The problem

Appendix

- HotSpot Virtual Machine Garbage Collection Tuning Guide (Java SE 25)
- Package `java.util.stream` (Java SE 25 API)
- The `jmap` command (Java SE 25)
- Java Object Layout (JOL), OpenJDK Code Tools

References: "Dollar" classes

Appendix

- The Java Tutorials: Anonymous Classes
- JLS SE 25, 15.9.5: Anonymous Class Declarations
- JLS SE 25, 13.1: The Form of a Binary ($ names)
- JLS SE 25, 8.1.3: Inner Classes and Enclosing Instances

References: Object pooling

Appendix

- `Integer.valueOf(int)` (Java SE 25 API): the -128 to 127 cache
- JLS SE 25, 5.1.7: Boxing Conversion
- The Java Tutorials: Thread Pools
- Java Garbage Collection Basics: why short-lived objects are cheap

References: Compressed OOPs

Appendix

- Java HotSpot VM Performance Enhancements (Java SE 25): Compressed Ordinary Object Pointer
- The `java` command (Java SE 25): `-XX:+UseCompressedOops`

References: Compact object headers

Appendix

- JDK 25 Release Notes: JEP 519 Compact Object Headers
- The Arrival of Java 25 (Oracle Java blog)
- The `java` command (Java SE 25)

References: Stream API

Appendix

- `java.util.stream` package summary (Java SE 25): pipelines, laziness, low-level construction
- `Stream` (Java SE 25 API)
- The Java Tutorials: Aggregate Operations
- The Java Tutorials: Reduction

References: Lambdas

Appendix

- The Java Tutorials: Lambda Expressions
- JLS SE 25, 15.27: Lambda Expressions (15.27.2 effectively final, 15.27.4 evaluation)
- `LambdaMetafactory` (Java SE 25 API)
- Java 8: Lambdas, Part 1 (Oracle technical article)

References: Escape analysis

Appendix

- Java HotSpot VM Performance Enhancements (Java SE 25): Escape Analysis
- Escape Analysis in the HotSpot JIT Compiler (Oracle Java Magazine)
- The `java` command (Java SE 25): `-XX:+DoEscapeAnalysis`

References: Solutions

Appendix

- OpenJDK Code Tools: Java Microbenchmark Harness (JMH)
- OpenJDK Code Tools: Java Object Layout (JOL)
- JEP 519: Compact Object Headers
- The `jcmd` command (Java SE 25)
- The Java Tutorials: Thread Pools
- Eclipse Collections